

On-Premise Deployment Guide

From compiled code to a licensed hospital installation

Version 1.0 · October 2026 · Internal and hospital IT use

THE JOURNEY AT A GLANCE

Part	Who	Where	How often
1. One-time platform setup	Polynexus	Dev machine + SaaS server	Once, ever
2. Build the compiled images	Polynexus	Build machine	Every release
3. Ship the images	Polynexus	Build machine	Every release
4. Prepare the hospital server	Hospital IT	Hospital server	Once per hospital
5. Issue the license	Polynexus	SaaS console	Once a year per hospital
6. Install and activate	Hospital IT	Hospital server	Once per hospital
7. Verify	Both	Browser	After install
8. Day-to-day: renewals, upgrades, backups	Both		Ongoing

Before you start

This guide takes you from the source code to a hospital running the system on its own server, with compiled (unreadable) code and a signed license. Follow the parts in order the first time.

What "compiled" means here:

- **Backend:** every Python file under `apps/` and `config/` becomes a native `.so` library with Cython. The `.py` source is deleted from the image.
- **Frontend:** a minified and obfuscated JavaScript bundle, with no source maps.

Licensing is enforced inside that compiled code, so a hospital can't edit it out.

1 One-time platform setup

1.1 Create the license signing key pair (done)

This was run once on 10 Oct 2026:

```
cd Backend
venv\Scripts\python.exe manage.py generate_license_keypair --private-key-out
C:\Users\User\Desktop\key\license_signing_key.pem
```

- **Public key:** now embedded in `Backend/apps/licensing/public_key.py`. **Commit this file.** Every on-premise image is built with it, and an image built without it refuses to build.
- **Private key:** `license_signing_key.pem`. Back it up to a password manager or vault. **Never commit it, never send it to a hospital.**

Do **not** run `generate_license_keypair` again. A new key pair makes every license already issued invalid.

1.2 Give the SaaS server the private key

The SaaS backend signs licenses, so it needs the private key. Add **one** of these to the SaaS server's environment (or `Backend/.env` when testing locally):

```
LICENSE_SIGNING_KEY_PATH=/secure/path/license_signing_key.pem
```

or paste the PEM itself, with line breaks written as `\n`:

```
LICENSE_SIGNING_KEY=-----BEGIN PRIVATE KEY-----\nMC4CAQAw...\n-----END PRIVATE KEY-----
```

1.3 Update the SaaS server's database

```
python manage.py migrate
```

This adds the `OnPremiseLicense` table, which holds the history of issued licenses. Restart the SaaS backend afterwards.

1.4 Check that signing works

Log in to the SaaS console → **Tenants & Subscriptions** → any tenant → **License**. If the key isn't configured, **Generate** shows "*License signing isn't configured*". Fix 1.2 if so.

2

Build the compiled images

Do this on any machine with Docker: Linux, or Windows/macOS with Docker Desktop running. Pick a version number for the release, e.g. `1.0.0`.

CPU architecture matters. Compiled `.so` files only run on the CPU type they were built for. Hospital servers are almost always `x86_64`, so always build with `--platform linux/amd64` (required on Apple-silicon Macs, harmless elsewhere).

2.1 Backend (Cython-compiled)

From the repository root:

```
docker build --platform linux/amd64 -f Backend/Dockerfile.backend.compiled -t
registry.polynexus.in/hms/hms-backend:1.0.0 Backend
```

The build:

1. installs the dependencies;
2. **stops** if `public_key.py` has no key (Part 1.1);
3. pins the image to on-premise mode, so `DEPLOYMENT_MODE=saas` can't switch licensing off;
4. compiles `apps/` and `config/` to `.so` and deletes all `.py` files except empty `__init__.py`;
5. **stops** if any application `.py` file is left over.

Expect it to take 10–20 minutes the first time. Cython compiles hundreds of modules.

Check the result: you should see `.so` files and no `.py` source.

```
docker run --rm --entrypoint sh registry.polynexus.in/hms/hms-backend:1.0.0 -c "find apps -name
'*.so' | head -3; echo; find apps config -name '*.py' ! -name '__init__.py' | wc -l"
```

The last line must print `0`.

Check that the compiled code runs:

```
docker run --rm --entrypoint python -e DJANGO_SETTINGS_MODULE=config.settings.dev -e
CACHE_URL=locmem:// registry.polynexus.in/hms/hms-backend:1.0.0 -c "import django;
django.setup(); from django.core.management import call_command; call_command('check');
print('ok')"
```

(It uses dev settings because production settings refuse to start without real secrets.) If this fails with an import error, the message names the module Cython couldn't handle. Report it, and that module can be left uncompiled.

2.2 Frontend (minified + obfuscated)

```
docker build --platform linux/amd64 -f Frontend/Dockerfile.frontend -t
registry.polynexus.in/hms/hms-frontend:1.0.0 Frontend
```

The build compiles TypeScript, builds with Vite (no source maps), obfuscates every JavaScript chunk, and packages it with nginx. nginx also forwards `/api/` to the backend.

The build deliberately ignores `Frontend/.env`. SIP/WebRTC telephony settings (`VITE_SIP_*`) are not baked into on-premise images. Add them as build args if a hospital uses the softphone.

2.3 Optional: a full trial run before shipping

Run Parts 4–7 on a spare **Linux** VM first, using `"*"` (any machine) as the fingerprint in Part 5. This catches problems before a hospital sees them. Test machine binding on the kind of Linux server hospitals use, not on Docker Desktop.

3

Ship the images

Choose the option that fits the hospital's network.

Option A: private registry (hospital server has internet)

```
docker login registry.polyplexus.in
docker push registry.polyplexus.in/hms/hms-backend:1.0.0
docker push registry.polyplexus.in/hms/hms-frontend:1.0.0
```

Give the hospital read-only registry credentials. They run `docker login registry.polyplexus.in` once on the server.

Option B: offline file (no internet, or no registry)

Bundle all four images (yours plus PostgreSQL and Redis) into one file:

```
docker pull --platform linux/amd64 postgres:16-alpine
docker pull --platform linux/amd64 redis:7-alpine
docker save -o hms-images-1.0.0.tar ^
  registry.polyplexus.in/hms/hms-backend:1.0.0 ^
  registry.polyplexus.in/hms/hms-frontend:1.0.0 ^
  postgres:16-alpine redis:7-alpine
```

(On Linux/macOS, use `\` instead of `^` for line continuation.)

Deliver `hms-images-1.0.0.tar` together with the `deploy/onprem/` **folder** (USB drive, secure file transfer).

The `deploy/onprem/` folder has everything the hospital needs and nothing more:

`docker-compose.onprem.yml` , `install.sh` , `env.template` , `README.md` . Never send the source code or the private key.

4

Prepare the hospital server

Server requirements: 64-bit Linux (Ubuntu 22.04/24.04 LTS recommended), 4+ CPU cores, 8+ GB RAM, 100+ GB disk, Docker Engine with the Compose v2 plugin, `openssl` .

```
# Ubuntu: install Docker
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER      # log out and back in afterwards
docker compose version             # must print v2.x
```

Copy the `deploy/onprem/` folder to the server, e.g. `/opt/hms` , then:

```
cd /opt/hms
chmod +x install.sh
docker load -i hms-images-1.0.0.tar # Option B only
```

Set the version you shipped (Part 2). `install.sh` creates `.env` on its first run. Afterwards, edit `HMS_VERSION=1.0.0` in `/opt/hms/.env` . Until then it defaults to `latest` , so you can also tag your images `latest` for the first install.

5

Issue the license

5.1 Hospital IT: get the machine fingerprint

Run the installer. On a server with no license it stops after printing the fingerprint:

```
cd /opt/hms
./install.sh
```

1. Enter the hostname or IP staff will use (e.g. `hms.cityhospital.local` or `192.168.1.50`). This creates `.env` with fresh random secrets.
2. The images are checked and PostgreSQL and Redis start.
3. The fingerprint prints, a 64-character code like:

```
==> This server's machine fingerprint
84f5a91df466b5c0d4cd4fe620faa45acb9b336c2f8cc53186eab930a5182b3c
```

4. Press **Enter** at the license prompt to stop for now.

Send the fingerprint to Polynexus.

The fingerprint comes from `/etc/machine-id` and the system UUID of **this server**. Reinstalling the OS or moving to new hardware changes it, and then a new license is needed.

5.2 Polynexus: create the tenant (first time only)

Each on-premise hospital needs a record in the SaaS console, because the license is issued for that hospital's ID. SaaS console → **Tenants & Subscriptions** → + **Onboard new hospital tenant**. Use the hospital's real name, which appears in the license. The hospital never logs into the SaaS platform; this record exists for licensing.

5.3 Polynexus: generate the license

SaaS console → **Tenants & Subscriptions** → the hospital's row → **License**:

Field	What to enter
Machine fingerprint	The 64-character code from 5.1. Tick Any machine only for cloud VMs that auto-scale or for a trial.
Expires on	Usually one year from today
Grace period	Days after expiry before the system goes read-only (default 14)
Max active users / Max beds	From the contract; 0 = unlimited
Tier	starter / pro / enterprise
Licensed modules	Tick what the hospital bought. Unticked modules are hidden and blocked.

Click **Generate & Download License**. A file like `LIC-2026-CITY-HOSPITAL-001.lic` downloads, and the license appears under **Issued licenses**, where you can download it again or revoke it.

Send the `.lic` file to the hospital. It is safe to email: it's signed, so it can't be edited, and it only works on that server.

6

Install and activate

Hospital IT, on the server:

```
cd /opt/hms  
./install.sh
```

This time, at the license prompt, give the path to the `.lic` file (e.g. `/home/admin/LIC-2026-CITY-HOSPITAL-001.lic`). The script then:

1. copies it to `license/hospital.lic` ;
2. creates the database tables (`migrate`);
3. runs `setup_onprem` , which verifies the license (signature, this machine, dates) and creates the hospital with its licensed modules. It then asks for the **owner account's email and password**: the first login, with full admin rights;
4. starts everything: `backend` , `worker` , `frontend` , `postgres` , `redis` .

If the license is rejected, the message says why: *"issued for a different server"* (wrong fingerprint, go back to Part 5), *"signature is not valid"* (file damaged; download it again), or a clock problem (fix the server time).

Back up `/opt/hms/.env` **now**. It holds the encryption keys for patient data. Losing it means losing the data.

7

Verify

1. Open `http://<hostname-from-6>/` and sign in with the owner account.
2. Go to **Settings** → **License**. It should show **Active**, the expiry date, limits, licensed modules and this server's fingerprint.
3. Only licensed modules appear in the menu.
4. Create a test record (e.g. a department), to confirm the system is writable.
5. Optional checks from the server:

```
docker compose -f docker-compose.onprem.yml ps          # all services "running"/"healthy"  
docker compose -f docker-compose.onprem.yml logs backend --tail 50
```

Then the hospital admin creates staff accounts and roles in **Admin**.

8

Renewals, upgrades, backups

Renewing the license (yearly)

From 30 days before expiry, hospital admins see a yellow banner. After expiry it turns red during the grace period, and then the system becomes **read-only**: records can be viewed but nothing can be saved.

1. Polynexus issues a new license (Part 5.3) with the **same fingerprint**. It's shown in Settings → License and under Issued licenses.
2. The hospital admin opens **Settings** → **License** and drops the new `.lic` file onto the upload box. It takes effect immediately, with no restart. (Or replace `license/hospital.lic` and run `docker compose -f docker-compose.onprem.yml restart backend worker`.)

An older license can't replace a newer one.

Upgrading to a new version

1. Build and ship the new images (Parts 2–3) with a new version, e.g. `1.1.0`.
2. On the server:

```
cd /opt/hms
docker load -i hms-images-1.1.0.tar          # or: docker compose -f
docker-compose.onprem.yml pull
sed -i 's/^HMS_VERSION=.*HMS_VERSION=1.1.0/' .env
docker compose -f docker-compose.onprem.yml up -d
```

Database migrations run automatically when the backend starts. The license, data and settings are kept.

Moving to a new server

The fingerprint changes. Install on the new server (Part 4–6 with the same `.env` and a copy of `pgdata/`), send the **new** fingerprint, and issue a new license. Revoke the old one in the SaaS console for your records.

Revoking only stops re-downloads. An offline server can't be reached, so a revoked license keeps working there until it expires or a newer one replaces it.

Backups (hospital IT)

What	Where	Why
<code>.env</code>	<code>/opt/hms/.env</code>	Encryption keys. Without them the data can't be read.
Database	<code>docker compose -f docker-compose.onprem.yml</code> <code>exec postgres pg_dump -U hms hms ></code> <code>backup-\$(date +%F).sql</code>	All hospital data. Schedule it daily.
Uploaded files	<code>/opt/hms/media/</code>	Documents, signatures
License	<code>/opt/hms/license/hospital.lic</code>	Can also be downloaded again from Polynexus

Troubleshooting

Symptom	Cause	Fix
Backend keeps restarting; logs say <i>"bound to a different server"</i>	Hardware or OS changed, or the license is for another server	Send the fingerprint shown in the log; issue a new license
Red banner <i>"read-only archive mode"</i>	License expired after the grace period	Renew (Part 8)
<i>"The system clock has been set back"</i>	Server time is earlier than a time it already recorded	Fix the server clock (enable NTP: <code>sudo timedatectl set-ntp true</code>)
<i>"No license is installed"</i>	<code>license/hospital.lic</code> missing or empty	Upload in Settings → License, or re-run <code>./install.sh</code>
<i>"The license allows N active users"</i> when adding staff	User cap reached	Deactivate unused accounts, or get a license with a higher cap
A module is missing from the menu	Not in the license	Issue a license that includes it
<code>install.sh</code> : <i>"Image ... is missing"</i>	Images not loaded	<code>docker load -i hms-images-<version>.tar</code> , check <code>HMS_VERSION</code> in <code>.env</code>
<i>"License signing isn't configured"</i> in the SaaS console	SaaS server lacks the private key	Part 1.2
Docker build stops: <i>"public_key.py has no key"</i>	Key pair never generated or not committed	Part 1.1

Reference

Item	Location
Licensing code	Backend/apps/licensing/
Compiled backend build	Backend/Dockerfile.backend.compiled , Backend/scripts/compile_cython.py
Obfuscated frontend build	Frontend/Dockerfile.frontend , Frontend/nginx.onprem.conf
Hospital-side kit	deploy/onprem/
Useful commands (on the server, inside <code>/opt/hms</code>)	<code>docker compose -f docker-compose.onprem.yml run --rm backend python manage.py get_machine_fingerprint</code>
License status API	GET <code>/api/v1/licensing/status/</code>